Windows preview

YOUR CODE. YOUR AGENT. YOUR STUDIO.

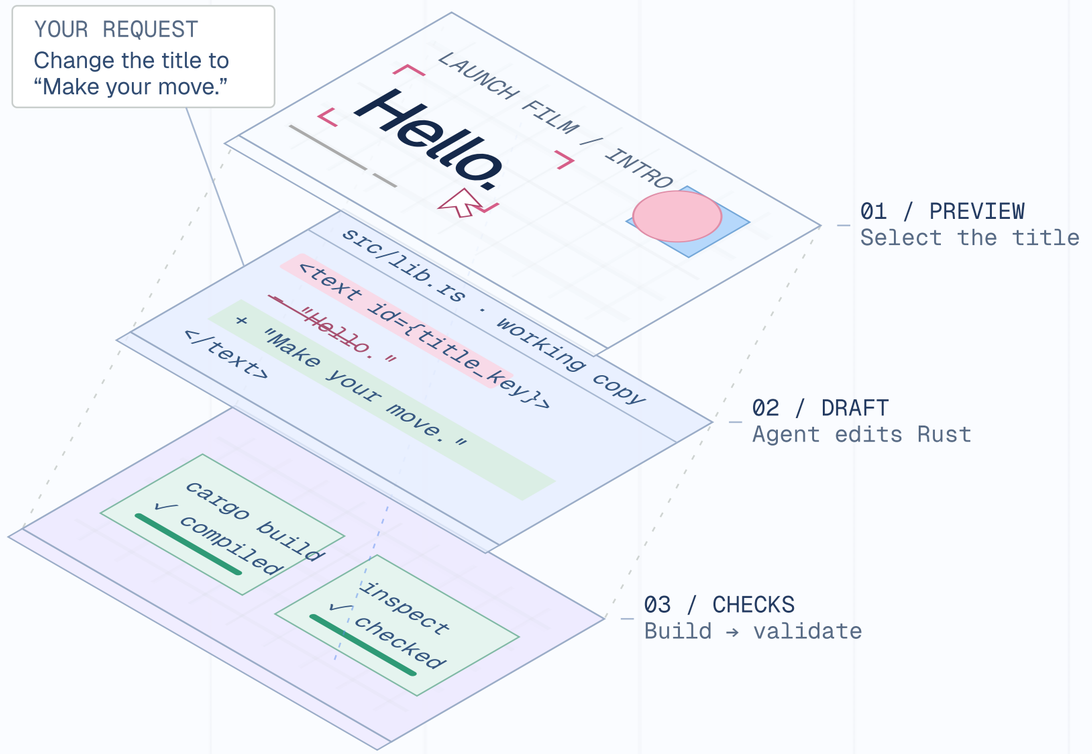
A video studio. Built around your code.

Describe an animated video to your AI coding assistant. Studio builds a preview you can watch. Select a scene, ask for a change, then export the finished video.

Requires your own AI assistant, such as Claude Code or Codex.

ANATOMY OF AN EDIT

ILLUSTRATION · NOT A LIVE APP



SOURCE → CANDIDATE → ACCEPTED REVISION

01–04 Selection → draft → checks → preview

01 Select 02 Edit 03 Check 04 Preview

ONE WAY TO WORK / THE AGENT LOOP

A small change.
A complete loop.

For example: ask for a short title video, then select its title and say “Change the title to ‘Make your move.’” You can also select a scene or time range when individual objects cannot be selected.

01 / SELECT

Start with what you can see.

Click a selectable object in the preview. Studio pauses at that frame and carries the selection into your next agent task.

> A selection gives context; it does not change the source.

02 / AGENT EDIT

Say what should change.

“Change the title to ‘Make your move.’” Your agent edits the Rust project in a draft working copy.

> The last working preview stays visible while the next revision takes shape.

03 / BUILD & CHECK

Put the draft through its paces.

Studio builds the candidate, then validates it with frame inspection and audio checks.

> A failed or cancelled build keeps the previous preview.

04 / UPDATED PREVIEW

Meet the next revision.

A validated edit applies automatically by default. With manual review, it waits for you to choose Apply.

> Your video remains an ordinary Rust project.

BRING YOUR OWN AGENT

Connect the coding agent you already use.

Claude Code	OpenAI Codex	OpenCode	Antigravity	Pi	ACP agents

CLAUDE CODE

Claude Code’s native stream-json wire.

CODEX

Codex’s app-server over JSON-RPC.

ACP

The Agent Client Protocol for other agents, such as Pi and Antigravity.

Sign-in stays with each agent. Studio stores no passwords or tokens.

THE REST OF YOUR WORKSPACE

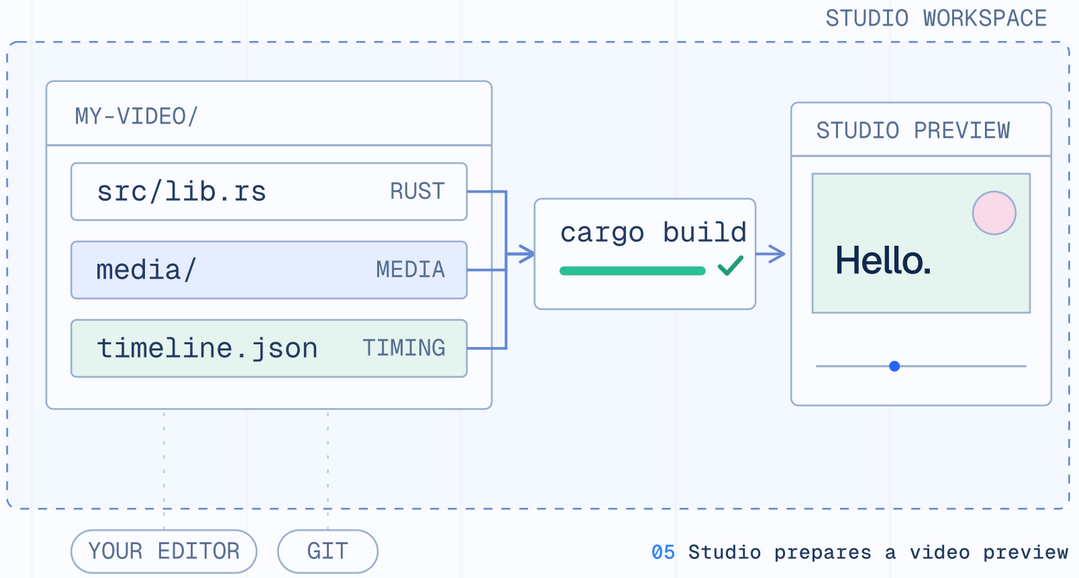
01 / YOUR PROJECT

Open files. Keep control.

Your video is a Rust project: source, media and a timing document. Open it in your editor, keep it in Git, and bring the tools you already use.

> Studio adds a workspace around those files. The project remains yours.

YOUR FILES, CONNECTED — ILLUSTRATION



A project structure, not a hidden document format. The preview requires a compatible project and a successful build.

THE REST OF YOUR WORKSPACE

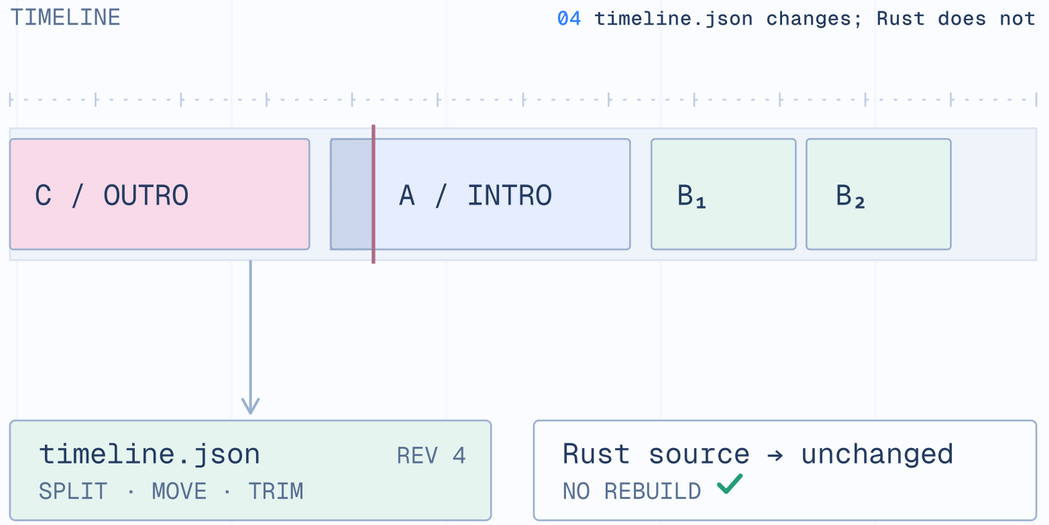
02 / THE TIMELINE

Change the rhythm. Keep the source.

Split a scene, move the outro, or trim a clip’s edge. Timing edits write timeline.json and update the preview without rebuilding Rust.

> Timing edits support undo and redo. Finish or cancel an agent task before changing the timeline.

CHANGE THE CUT, KEEP THE CODE ILLUSTRATION



Illustrative split → reorder → trim. Timing-only edits do not rebuild Rust. Timing changes are unavailable while an agent task is running.

THE REST OF YOUR WORKSPACE

03 / THE BUILD ENVIRONMENT

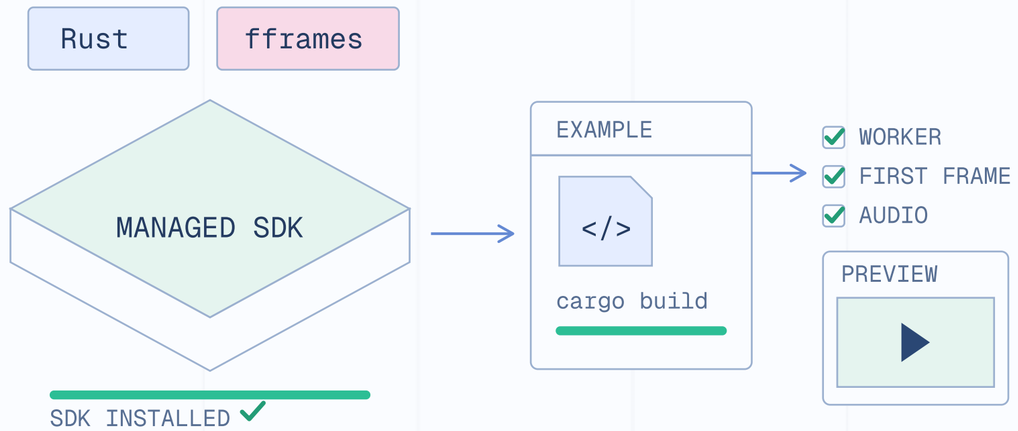
An example in. A preview out.

Install the managed SDK from Setup, then open an example. Studio builds the project and prepares its worker, first frame and audio.

> The current preview stays available while the next revision is prepared.

FROM SETUP TO A WORKING PREVIEW

ILLUSTRATION



05 The new preview replaces the current one

A preparation sequence, not elapsed build time. Failed or cancelled builds keep the previous preview.

THE REST OF YOUR WORKSPACE

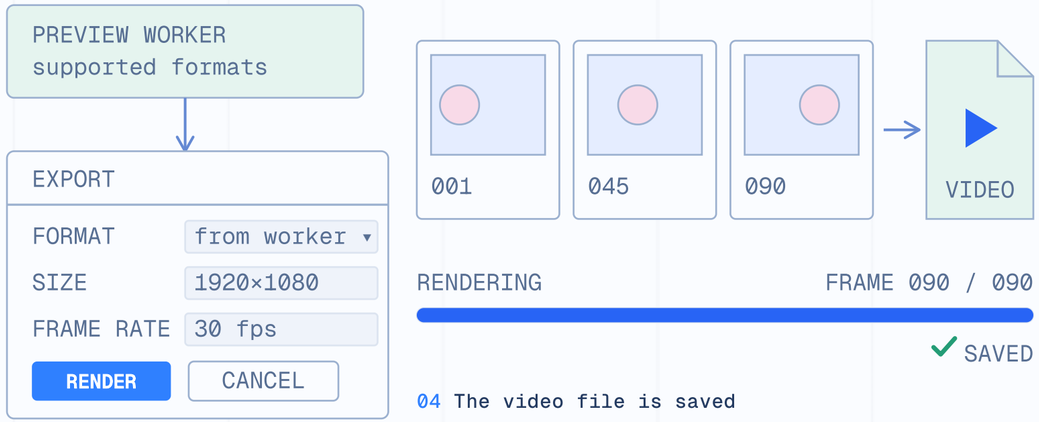
04 / THE FINISHED FILE

Make the video. Take it with you.

Choose an export format supported by your project's worker, set the size and frame rate, and render a video file. Follow progress or cancel the export.

> From editable source to a file you can share.

TURN THE PROJECT INTO A VIDEO — ILLUSTRATION



Illustrative frames and render progress. Available formats, size and frame rate come from the project's worker.

START WITH AN EXAMPLE.

Make your first move.

01 Install Studio

02 Open an example

03 Make it yours

INSTALL WITH POWERSHELL · PER USER, NO ADMINISTRATOR RIGHTS

```
PS>  irm https://github.com/khanhthanhdev/fframes-desktop/releases/latest/download/install.ps1 | iex
```

FROM A CHECKOUT · THEN INSTALL THE SDK ON THE SETUP SCREEN

```
$  cargo run --locked --manifest-path desktop/Cargo.toml -p fframes-studio
```

STATUS Windows is the tested platform and the preview is unsigned. Linux runs the tests in CI. macOS is not fully tested yet.

INSIDE FFRAMES STUDIO

An agent edits. Studio checks.

A change starts as a draft. Follow the checks that stand between an agent's edit and the revision you see.

THE OPERATING PRINCIPLE

Your working preview stays available while Studio prepares the next revision.

THE FFRAMES RENDERING ENGINE

From Rust code to a finished video.

01 / DESCRIBE EACH FRAME

At each point in time, `render_frame` describes the text, shapes, images and their positions. The `svgr!` macro builds an SVG-style tree of drawing instructions.

02 / DRAW THE PICTURE

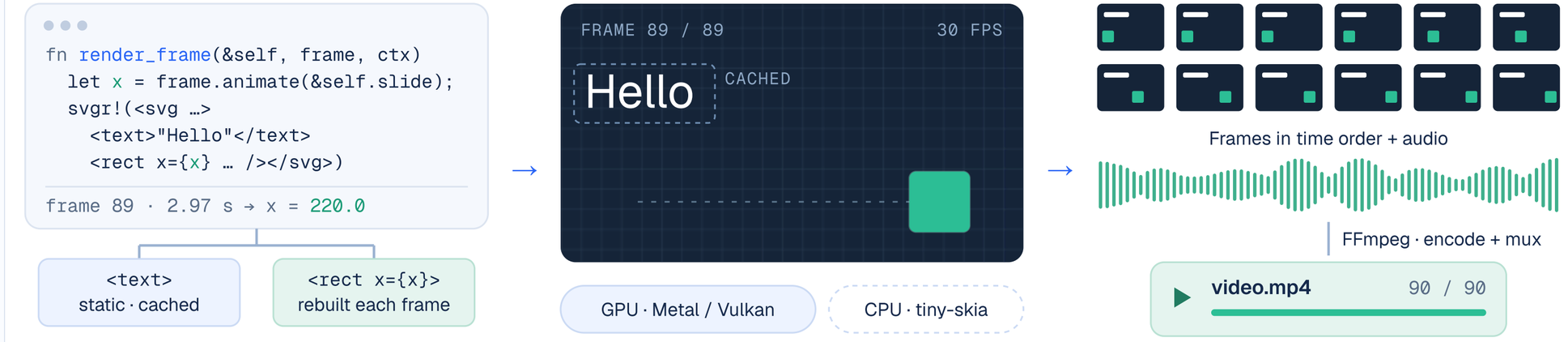
Skia draws the tree into a frame on the GPU (Metal or Vulkan). Without a usable GPU, tiny-skia draws on the CPU, without shader effects.

03 / SAVE THE VIDEO

FFmpeg encodes the frames and combines them with the audio into one video file.

ONE ANIMATION, ONE FRAME AT A TIME

SCROLL TO ADVANCE THE FRAME · ILLUSTRATION



A tree of drawing instructions, not pixels yet.

This frame's x becomes pixels.

Pictures and sound become one file.

A CONNECTED WORKSPACE

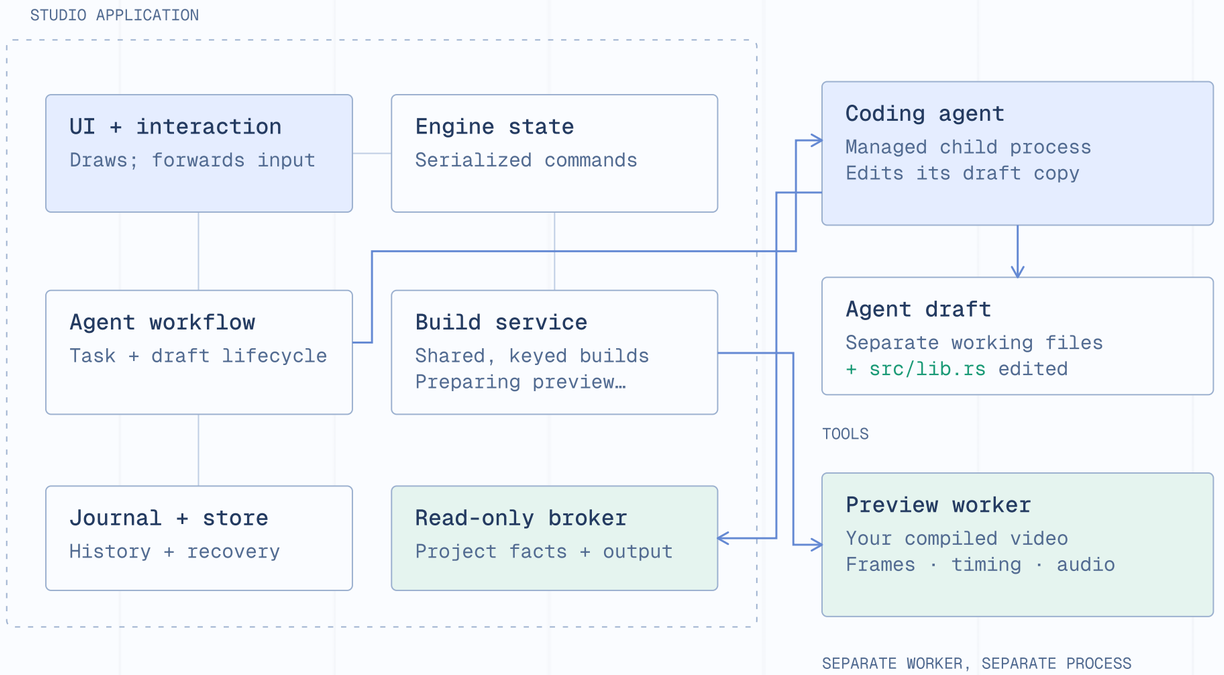
Separate processes. Clear responsibilities.

The app coordinates the work. Your coding agent edits a draft in its own process. Your video runs in a separate preview worker.

> If the video's code panics or crashes, the app survives and keeps the previous preview. This is crash isolation, not a security sandbox.

WHO DOES WHAT

ILLUSTRATION



05 The build service prepares the preview worker

Responsibility map, not a complete call graph. The agent owns draft edits; the preview worker remains separate from the Studio UI.

01 / THE EDIT TRANSACTION

A draft goes in. A revision comes out.

01 / CAPTURE

After the agent's writer is quiet, Studio captures the draft twice. Both captures must match.

02 / BUILD

Compile in an isolated copy bound to the project's managed SDK.

03 / INSPECT

Frames, range boundaries and audio are checked. A failure triggers at most one automatic repair.

04 / REVIEW POLICY

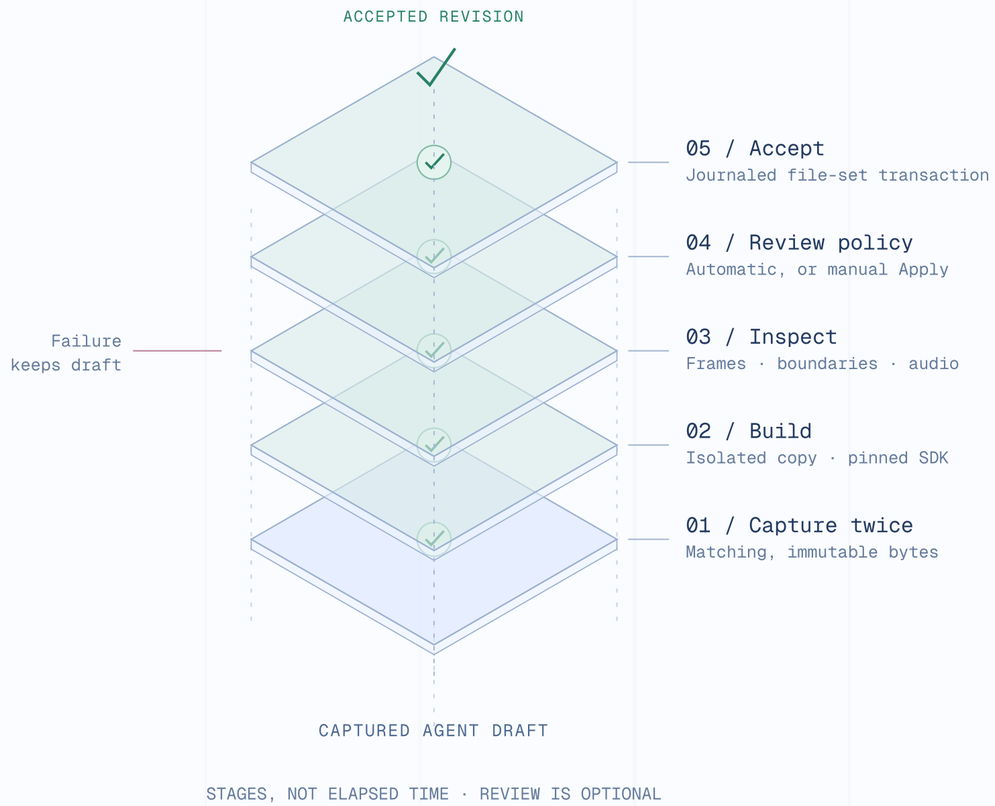
Applied automatically by default. Manual review waits for Apply and can keep a subset of files.

05 / ACCEPT

A journaled file-set transaction publishes the revision. Undo passes validation too.

THE CANDIDATE'S PATH

ILLUSTRATION · SUCCESSFUL PATH



STAGES, NOT ELAPSED TIME · REVIEW IS OPTIONAL

01 Capture 02 Build 03 Inspect 04 Review 05 Accept

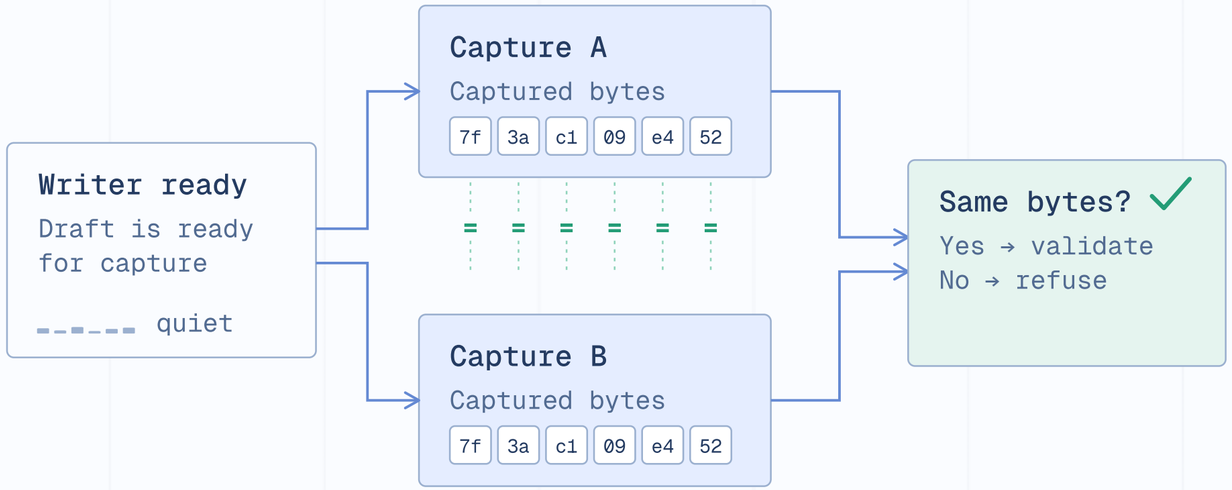
WHY CAPTURE TWICE?

“Agent finished” is not “video ready”.

Two captures are compared after the writer is quiet. If they differ, the working copy is still changing, and it never becomes the candidate that is validated.

> A compile success, cached or not, cannot replace this capture check.

WHY CAPTURE TWICE?
ILLUSTRATION



A CHANGING DRAFT DOES NOT PROCEED

05 Match → validate

Two captures are compared after writer quiescence. A compile success cannot replace this capture check.

02 / GROUNDING

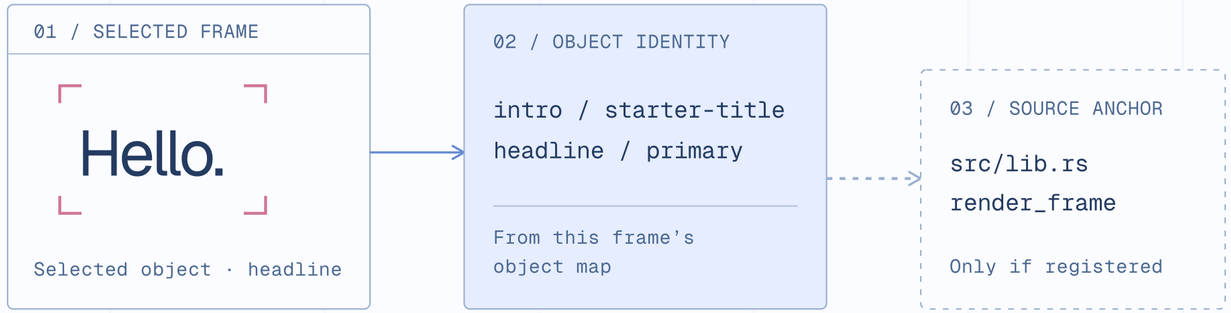
From a visible object to useful context.

A click uses the object map attached to that exact frame. Topmost paint order wins; cycling reveals overlapping objects.

```
let title_key = EditorObjectKey::new(
    scene_key, "starter-title", "headline", "primary",
)?.with_source_anchor(
    "src/lib.rs", "render_frame", Some("title-marker"),
)?;
```

THE SELECTION BECOMES CONTEXT

ILLUSTRATION



SELECTION PROVIDES CONTEXT. IT DOES NOT EDIT SOURCE.

03 A registered anchor can add a source location

Frame → stable object identity → optional source anchor. Without a registered anchor, Studio does not infer a source location.

03 / PROJECT TOOLS

A way to look.
Nine read-only tools.

`project_context`
Project facts

`source_lookup`
Verified source snippets

`render_frame`
A rendered frame

`inspect`
Frame diagnostics

`build_status`
Build status

`selection_context`
Frozen task scope

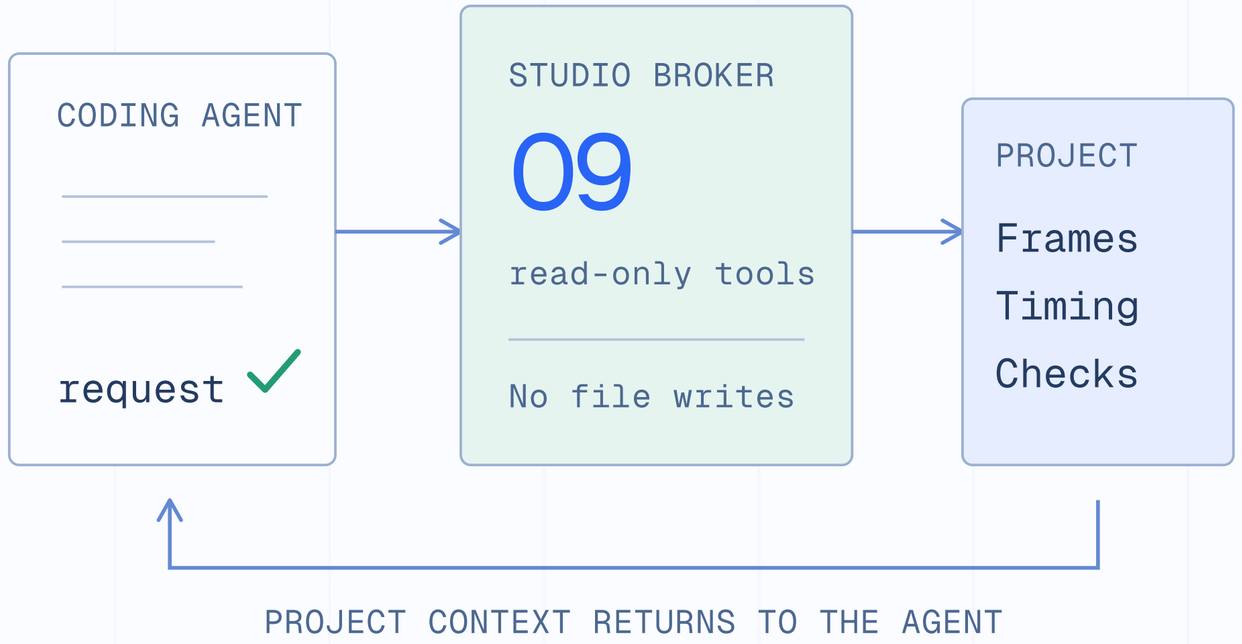
`timeline`
Scene timing

`render_strip`
A frame contact sheet

`style_context`
Style context

> Studio's broker provides no shell execution, file writes or export. The agent's own tools can still edit files, subject to your permission policy.

LOOK THROUGH THE BROKER
ILLUSTRATION



04 The answer returns to the agent

Read-only applies to Studio's broker. The agent's own tools can edit under your permission policy.

04 / PREVIEW READINESS

Prepare the whole revision. Then switch.

The current build keeps playing while the next one is prepared. Worker, frame and audio switch together at the playhead, once the revision is ready.

ONE COORDINATED HANDOFF

ILLUSTRATION



The old revision remains available through compile and preparation. Readiness is more than a compiler success.

05 / WRITER CONTAINMENT

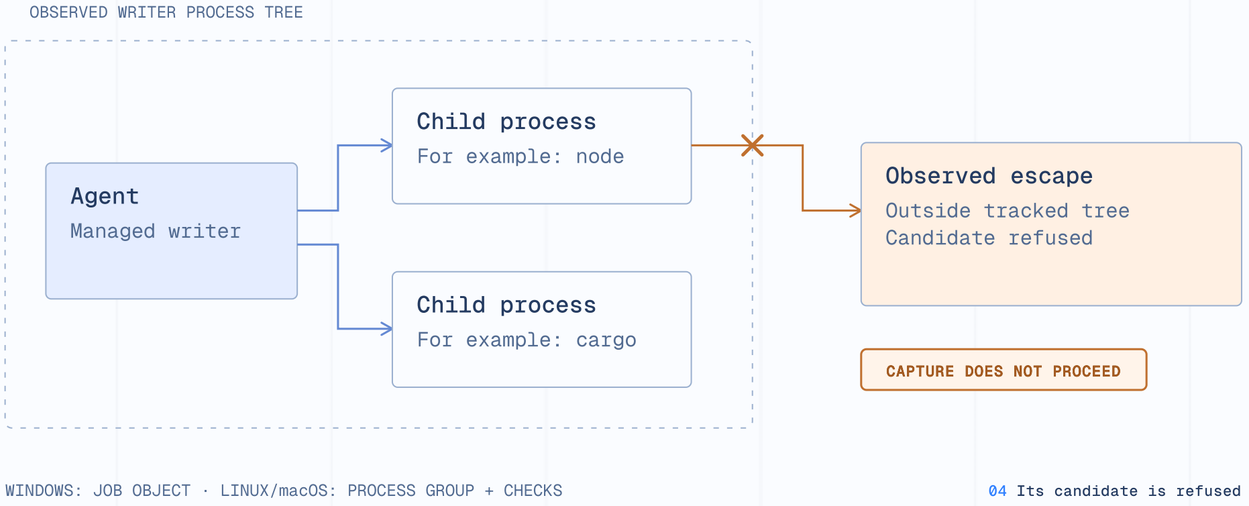
A process boundary. A condition for capture.

Windows uses a job object; Linux and macOS use a process group with escape checks. If Studio observes a writer escape, its candidate is refused.

> Containment is not a general sandbox. Work done by pre-existing services on an agent's behalf is outside the observed boundary.

OBSERVED ESCAPE → CAPTURE REFUSED

ILLUSTRATION



The dashed line is an observed process boundary, not a universal sandbox. Work delegated to pre-existing services can be outside this boundary.

06 / RECOVERY & TIMING

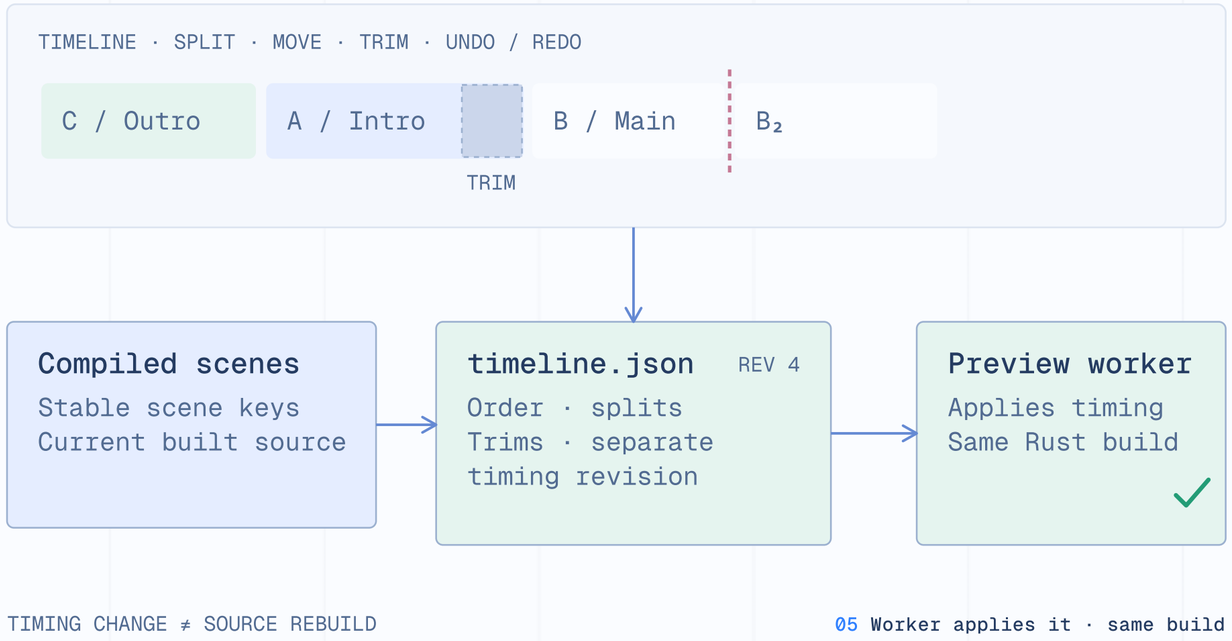
Keep the accepted revision. Retain the unfinished work.

After a crash, Studio recovers to the last accepted revision, marks unfinished work interrupted, and offers the retained draft. Process exit is not proof of a successful build or apply.

> Timing has its own document (`timeline.json`): timing-only changes reuse the current build; source changes still need compilation.

TIMING HAS ITS OWN DOCUMENT

ILLUSTRATION



Timing-only changes reuse the current build. Source changes still need compilation; timing edits are unavailable during an agent task.

FROM SOURCE TO FRAME

Your next idea.
Already in motion.

A native video workspace for your code, your coding agent, and your next idea.

- Windows preview available

`fframes-studio.pages.dev`

`github.com/khanhthanhdev/fframes-desktop`

`fframes library · github.com/dmtrKovalenko/fframes`



SCAN TO GET STUDIO